

Here is the raw markdown text for the Technical Wiki, ready for you to copy and paste directly into your own documentation.

```markdown

# Technical Wiki: Claude Prompt Caching for Agent CLI

### ## 1. Core Concepts & Mechanics

Prompt caching optimizes your API usage by allowing the system to resume from specific prefixes in your prompts [1]. This significantly reduces processing time and costs for repetitive tasks or prompts with consistent elements [1]. Cache prefixes are strictly created in the following hierarchical order: `tools`, `system`, then `messages` [2]. Caches are isolated per workspace, ensuring data separation between workspaces within the same organization [3].

There are two primary ways to enable prompt caching:

- \* \*\*Automatic caching:\*\* Add a single `cache\_control` field at the top level of your request [4]. The system automatically applies the cache breakpoint to the last cacheable block and moves it forward as conversations grow [4].
- \* \*\*Explicit cache breakpoints:\*\* Place `cache\_control` directly on individual content blocks for fine-grained control over exactly what gets cached [4]. You can define up to 4 explicit cache breakpoints per request [5].

### ## 2. Pricing & Token Multipliers

Prompt caching introduces specific pricing multipliers that stack with other modifiers, such as the Batch API discount [6]:

- \* \*\*Cache reads:\*\* 0.1x (10%) of the base input token price [6].
- \* \*\*5-minute cache writes:\*\* 1.25x of the base input token price [6].
- \* \*\*1-hour cache writes:\*\* 2.0x of the base input token price [6].
- \* \*\*Cache breakpoints:\*\* The breakpoints themselves do not add any extra cost [7].

### ## 3. The 20-Block Lookback Rule

When using explicit caching, the system relies on a 20-block lookback window [2]. On each request, the system computes the prefix hash at your breakpoint and checks for a matching cache entry [2]. If none exists, it walks backward one block at a time checking for a match, but it checks at most 20 positions [2]. If a growing conversation pushes your breakpoint 20 or more blocks past the last write, the lookback window misses it, and you will pay for a fresh write [5].

**\*\*Key Optimization Strategy:\*\*** Place `cache\_control` on the last block whose prefix is perfectly identical across requests [8]. If your prompt has a varying suffix (like timestamps, per-request context, or the incoming user message), place the breakpoint at the end of the static prefix, not on the varying block [8]. Marking a varying block means the hash will never match, causing a cache miss every time [9].

### ## 4. Time-To-Live (TTL) Management

By default, the cache uses a 5-minute lifetime [10]. This 5-minute cache is refreshed for no additional cost each time the cached content is used [10].

Anthropic also offers a 1-hour cache duration at additional cost, configured by including ``ttl`` in the ``cache_control`` definition [11, 12].

**\*\*The Mixing Rule:\*\*** You can use both 1-hour and 5-minute cache controls in the same request [13]. However, cache entries with a longer TTL must appear before shorter TTLs (a 1-hour entry must appear before any 5-minute entries) [13].

### ## 5. Pre-Warming the Cache (Latency Reduction)

Cache pre-warming allows you to load your system prompt or tool definitions into the prompt cache before a user triggers a real request, eliminating the cache-miss latency penalty on the first interaction [14].

**\*\*Implementation Rules:\*\***

- \* Set ``max_tokens: 0`` in your request [15].

- \* The API reads your prompt into the model and writes the cache at your breakpoint, returning immediately without generating any output [15].

- \* A ``max_tokens: 0`` request will be rejected if ``stream: true``, extended thinking, or structured outputs are enabled [16].

- \* Pre-warming with ``max_tokens: 0`` is completely rejected inside a Message Batches request because pre-warming targets time-to-first-token latency, which does not apply to background batch processing [16].

### ## 6. What Invalidates the Cache?

Modifications to cached content can invalidate some or all of the cache based on the hierarchy (``tools`` → ``system`` → ``messages``) [17].

**\*\*Tools:\*\*** Modifying tool definitions (names, descriptions, parameters) invalidates the entire cache (tools, system, and messages) [17, 18].

**\*\*System Prompt:\*\*** Enabling/disabling web search or citations only modifies the system prompt, keeping the tools cache valid but invalidating system and messages caches [18].

**\*\*Messages:\*\*** Changes to the ``tool_choice`` parameter, changing extended thinking budgets, or adding/removing images anywhere in the prompt will only invalidate the message blocks [18, 19].

### ## 7. Tracking Cache Diagnostics

To verify caching is working, track cache performance using these API response fields within the ``usage`` block [20]:

- \* ``cache_creation_input_tokens``: The number of tokens written to the cache when creating a new entry [20].

- \* ``cache_read_input_tokens``: The number of tokens retrieved from the cache for this request [20].

- \* ``input_tokens``: The number of input tokens which were not read from or used to create a cache (these are only the tokens that come after your last cache breakpoint) [20, 21].

